DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

(Artificial Intelligence & Machine Learning)

Academic Year:	2026-27	**Name of Student:**	
Semester:	V	**Student ID:**	
Class / Branch:	TE – CSE (AI&ML)	**Roll No:**	
Subject:	WMD Lab	**Date of Submission:**	

Experiment No. 09

Lab Outcome: Apply microservices, REST APIs, and deployment techniques for scalable applications.

Aim: Deploy Spring Boot app to a cloud platform (Render/Heroku) or local server.

Software used: Java JDK17 or above, Spring Boot, Maven, Git & GitHub Account, Render Account

Theory: -

1. What is Application Deployment?

Deployment is the process of taking an application from a developer's own machine and making it available for other people to actually use — either on a local network, or, more commonly, on the public internet via a cloud platform. A Spring Boot application that only "works on my laptop" has no real value to end users until it has been deployed somewhere they can reach it.

	Local Deployment	Cloud Deployment
Where it runs	The developer's own computer	A remote server managed by a cloud provider (e.g. Render)
Who can access it	Only the developer (via localhost)	Anyone with the public URL, from anywhere
Purpose in this	Verifies the application builds and runs	Demonstrates a real, internet-

	Local Deployment	Cloud Deployment
experiment	correctly before deployment	accessible deployment

2. Maven and the Build Lifecycle

Maven is a build automation and dependency management tool for Java projects. Instead of manually downloading libraries such as Spring Web, a project simply declares them in a file called pom.xml, and Maven downloads and manages them automatically. Maven also defines a standard build lifecycle — a sequence of phases such as compile, test, and package — that transforms raw source code into a runnable artifact.

Most Spring Initializr projects also include the Maven Wrapper (mvnw / mvnw.cmd) — a small script committed alongside the project that downloads the exact correct Maven version automatically. This is why the build command in this experiment is .\mvnw rather than a plain mvn: it guarantees the same Maven version is used regardless of what is (or is not) already installed on the machine.

3. Anatomy of a Spring Boot Application

Every Spring Boot application needs exactly one class annotated with @SpringBootApplication and a main() method that calls SpringApplication.run(...). This class is the entry point: it is what actually starts the embedded web server and brings the rest of the application to life. The Implementation section below explains this class, DemoApplication.java, and its annotations in detail.

4. Version Control with Git and GitHub

Git is a version control system that tracks changes to a project's source code over time. GitHub is a cloud-hosted platform for storing Git repositories online. Cloud platforms such as Render do not accept a ZIP file or a folder upload for deployment — instead, they connect directly to a GitHub repository and rebuild the application automatically whenever new code is pushed to it. This is why, in Part B of this experiment, the project must first be committed to Git and pushed to GitHub before Render can deploy it at all.

A Personal Access Token (PAT) is used in place of a GitHub password when pushing code from the command line. GitHub no longer accepts plain passwords for this purpose, so a PAT — a long, revocable, scope-limited credential — is generated once and used instead.

5. Containerization with Docker

Docker packages an application together with everything it needs to run — the operating system libraries, the Java runtime, and the compiled application itself — into a single, portable unit called a container image. Instead of a cloud platform having to guess how to build and run an arbitrary Java project, a Dockerfile gives it an exact, repeatable recipe to follow.

A multi-stage Dockerfile (used in this experiment) builds the application in two separate stages: a build stage, which contains Maven and the full JDK and is used only to compile the project into a .jar file, and a run stage, which starts from a much smaller image containing only the Java Runtime Environment (JRE) and copies in just the finished .jar file. This keeps the final deployed image small and secure, since none of the build tools are shipped to production.

6. Platform-as-a-Service (PaaS) Deployment

Render is a Platform-as-a-Service (PaaS): a cloud provider that manages the underlying servers, networking, and infrastructure, so a developer only has to supply the application code (and, in this case, a Dockerfile). When Docker is selected as the deployment method, Render automatically detects and uses the Dockerfile committed to the repository to build and run the container — no manual build or start commands need to be configured.

7. Environment-Based Configuration

Cloud platforms typically assign a web application's port dynamically at startup, through an environment variable named PORT, rather than always using a fixed port such as 8080. The property server.port=${PORT:8080} tells Spring Boot: "listen on whatever port the PORT environment variable specifies; if that variable is not set (as on a local machine), fall back to 8080 instead." This single line lets the exact same application run correctly both locally and on Render, without any code changes between the two environments.

Implementation

Part A — Deploy the Spring Boot Application on a Local Server

Step 1 — Create the Spring Boot Project

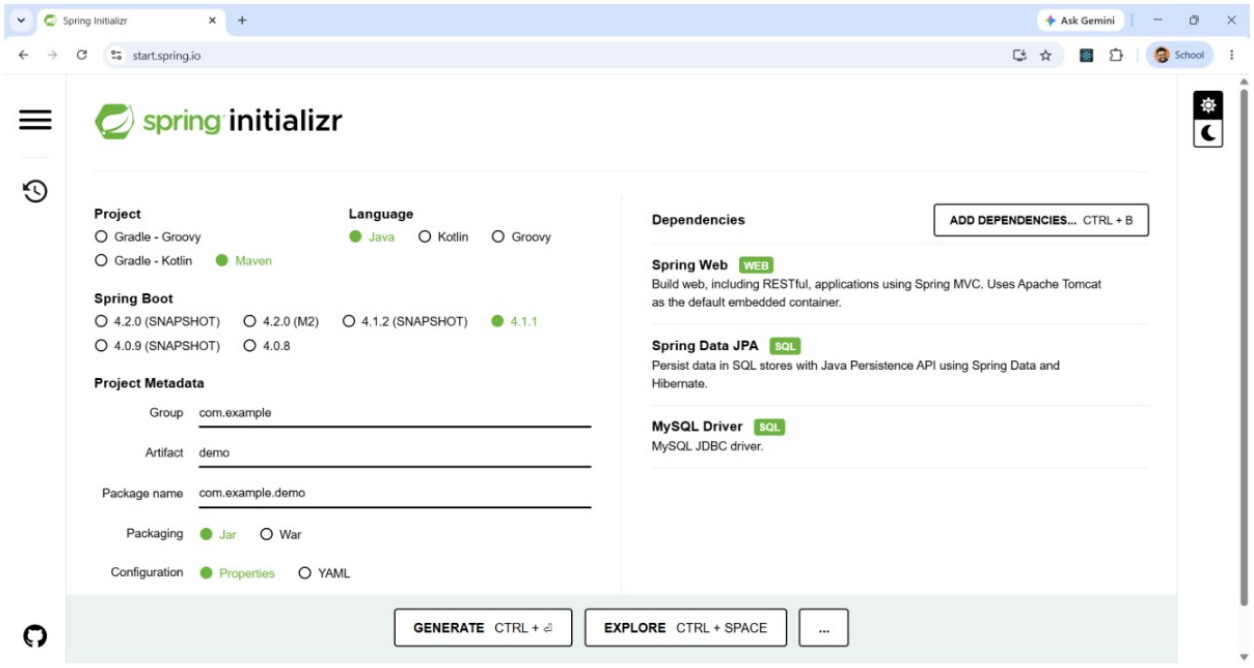
Spring Initializr (start.spring.io) is a project-generation tool: rather than assembling a pom.xml and folder structure by hand, a developer selects the desired options and downloads a ready-to-use skeleton project. This is the starting point for every Spring Boot application in this course.

- Visit: https://start.spring.io
- Select: Maven Project, Java, Spring Boot `4.1.1`, Packaging: Jar
- Dependencies: Spring Web, Spring Data JPA, MySQL Driver

Generate and download the project, then extract it to the working path:

```
D:\AY_2026-27\ODD\WMD_VJshNair\Lab\Exp-9\Program\
```

Note: although Spring Data JPA and the MySQL Driver dependencies are included (in case the project is later extended with a real database), this experiment does not connect to an actual MySQL server — application.properties (Step 3) deliberately switches off database auto-configuration, so the application can start and be deployed without requiring a live database.



Step 2 — Create a Simple REST Controller (HelloController.java)

A @RestController is the layer of a Spring Boot application that actually handles incoming HTTP requests and returns a response. For this experiment, the goal is simply to prove that the deployed application is reachable and running — so a single, minimal endpoint is enough: one that responds to a request for the root URL ("/") with a short confirmation message.

What this code does: it maps HTTP GET requests for "/" to a method that returns a plain text message. What comes next: this class needs to be discovered and registered by Spring at startup — that discovery process is handled by @ComponentScan inside the DemoApplication class covered in Step 3.

Why this file is important: without at least one controller, a Spring Boot web application has nothing to actually serve — visiting its URL would return only a default error page. HelloController.java is what turns "an application that starts" into "an application that responds", and its output ("Spring Boot Application Deployed Successfully") is exactly what is used in Step 6 to verify that the deployment worked.

Path:D:\AY_2026-27\ODD\WMD_VJshNair\Lab\Exp-9\Program\demo\src\main\java\com\example\demo\HelloController.java

D: > AY_2026-27 > ODD > WMD_VJshNair > Lab > Exp-9 > Program > demo > src > main > java > com > example > demo > HelloController.java >

```java
package com.example.demo;

import org.springframework.web.bind.annotation.*;

@RestController
public class HelloController {

    @GetMapping("/")
    public String home() {
        return "Spring Boot Application Deployed Successfully";
        }
}
```

Step 3 — The Application Entry Point (DemoApplication.java)

DemoApplication.java is the main entry point of a Spring Boot application. It contains the main() method required to start the entire application, launch the embedded server (Tomcat), and bootstrap the Spring framework. What this code does: it is the single class Java actually executes

when the application starts, and it is responsible for triggering everything else — auto-configuration, component scanning, and server startup.

The @SpringBootApplication Annotation

This single annotation combines three essential Spring Boot features:

- **@EnableAutoConfiguration:** tells Spring Boot to automatically configure the application based on the dependencies found in pom.xml — for example, setting up an embedded Tomcat web server as soon as Spring Web is detected on the classpath.
- **@ComponentScan:** tells Spring Boot to scan the package where DemoApplication is located (com.example.demo) and all of its sub-packages. This is how Spring finds and registers @RestController classes such as HelloController.java from Step 2.
- **@SpringBootConfiguration:** marks this class as a configuration source for Spring beans.

SpringApplication.run(DemoApplication.class, args)

When the application is executed (via java -jar demo.jar, or run locally from the IDE), this single method call:

- Initializes the Spring Application Context.
- Starts the embedded web server (Apache Tomcat, on port 8080 by default).
- Scans, instantiates, and wires together all Spring components and controllers.

Why this file is important: Executable JAR — when Maven packages the application using mvn clean package, the Spring Boot plugin looks for this class to set as the Main-Class inside the JAR manifest; without it, the JAR cannot run as a standalone application at all. Component Scanning Anchor — Spring searches for controllers and services starting from the folder where DemoApplication.java resides, so keeping it at the root package (com.example.demo) ensures every other class in the project is discovered automatically.

What comes next: once this class can start the server and discover HelloController, the application still needs to know which network port to listen on — that is configured in Step 4.

Path:D:\AY_2026-27\ODD\WMD_VJshNair\Lab\Exp-9\Program\demo\src\main\java\com\example\demo\DemoApplication.java

D: > AY_2026-27 > ODD > WMD_VJshNair > Lab > Exp-9 > Program > demo > src > main > java > com > example > demo > DemoApplication.java >

```java
package com.example.demo;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class DemoApplication {

    public static void main(String[] args) {
        SpringApplication.run(DemoApplication.class, args);
    }

}
```

Step 4 — Configure the Server Port (application.properties)

What this code does: it configures two aspects of the running application before it starts — which network port it listens on, and which auto-configuration features it should deliberately skip.

As explained in the Theory section, server.port=${PORT:8080} allows the same build to run correctly on a local machine (port 8080) and on Render (whatever port Render assigns via the PORT environment variable), with no code changes required. The second line switches off DataSourceAutoConfiguration and HibernateJpaAutoConfiguration — normally, having Spring Data JPA and a MySQL Driver on the classpath causes Spring Boot to try to auto-configure a real database connection at startup, which would fail immediately since no MySQL server exists in this demo. Disabling it lets the application start cleanly using only the REST controller from Step 2.

> *Why this file is important: without disabling this auto-configuration, the application would crash on startup with a "failed to configure a DataSource" error the moment it tries to deploy — this single file is what keeps the deployment demonstration simple and self-contained.*

What comes next: with the code complete, the project is ready to be compiled into a runnable artifact — that is the Build step.

Path:D:\AY_2026-27\ODD\WMD_VJshNair\Lab\Exp-9\Program\demo\src\main\resources\application.properties

D: > AY_2026-27 > ODD > WMD_VJshNair > Lab > Exp-9 > Program > demo > src > main > resources > application.properties

```
server.port=${PORT:8080}
spring.autoconfigure.exclude=org.springframework.boot.jdbc.autoconfigure.DataSourceAutoConfiguration,org.springframework.boot.hibernate.autoconfigure.HibernateJpaAutoConfiguration
```

```
server.port=${PORT:8080}
spring.autoconfigure.exclude=org.springframework.boot.jdbc.autoconfigure.DataSourceAutoConfiguration,org.springframework.boot.hibernate.autoconfigure.HibernateJpaAutoConfiguration
```

Step 5 — Build the Project

The package phase of the Maven build lifecycle compiles all the Java source code and bundles it — along with the embedded Tomcat server and all dependencies — into a single, self-contained, executable .jar file. -DskipTests tells Maven to skip running the project's automated tests during this build, which speeds up the process for a quick classroom demonstration.

Open a terminal in the project folder and run:

```
.\mvnw clean package -DskipTests
```

```
PS D:\AY_2026-27\ODD\WMD_VJshNair\Lab\Exp-9\Program\demo> .\mvnw clean package -DskipTests
[INFO] Scanning for projects...
[INFO]
[INFO] ----------------------< com.example:demo >----------------------
[INFO] Building  0.0.1-SNAPSHOT
[INFO]   from pom.xml
[INFO] --------------------------------[ jar ]---------------------------------
[INFO]
[INFO] --- clean:3.5.0:clean (default-clean) @ demo ---
[INFO] Deleting D:\AY_2026-27\ODD\WMD_VJshNair\Lab\Exp-9\Program\demo\target
[INFO]
[INFO] --- resources:3.5.0:resources (default-resources) @ demo ---
[INFO] Copying 1 resource from src\main\resources to target\classes
[INFO] Copying 0 resource from src\main\resources to target\classes
[INFO]
[INFO] --- compiler:3.15.0:compile (default-compile) @ demo ---
[INFO] Recompiling the module because of changed source code.
[INFO] Compiling 2 source files with javac [debug parameters release 17] to target\classes
[INFO]
[INFO] --- resources:3.5.0:testResources (default-testResources) @ demo ---
[INFO] skip non existing resourceDirectory D:\AY_2026-27\ODD\WMD_VJshNair\Lab\Exp-9\Program\demo\src\test\resources
[INFO]
[INFO] --- compiler:3.15.0:testCompile (default-testCompile) @ demo ---
[INFO] Recompiling the module because of changed dependency.
[INFO] Compiling 1 source file with javac [debug parameters release 17] to target\test-classes
[INFO]
[INFO] --- surefire:3.5.6:test (default-test) @ demo ---
[INFO] Tests are skipped.
[INFO]
[INFO] --- jar:3.5.1:jar (default-jar) @ demo ---
[INFO] Building jar: D:\AY_2026-27\ODD\WMD_VJshNair\Lab\Exp-9\Program\demo\target\demo-0.0.1-SNAPSHOT.jar
[INFO]
[INFO] --- spring-boot:4.1.1:repackage (repackage) @ demo ---
[INFO] Replacing main artifact D:\AY_2026-27\ODD\WMD_VJshNair\Lab\Exp-9\Program\demo\target\demo-0.0.1-SNAPSHOT.jar with repackaged archive, adding nested dependencies in BOOT-INF/.
[INFO] The original artifact has been renamed to D:\AY_2026-27\ODD\WMD_VJshNair\Lab\Exp-9\Program\demo\target\demo-0.0.1-SNAPSHOT.jar.original
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  8.055 s
[INFO] Finished at: 2026-09-29T23:36:54+05:30
[INFO] ------------------------------------------------------------------------
```

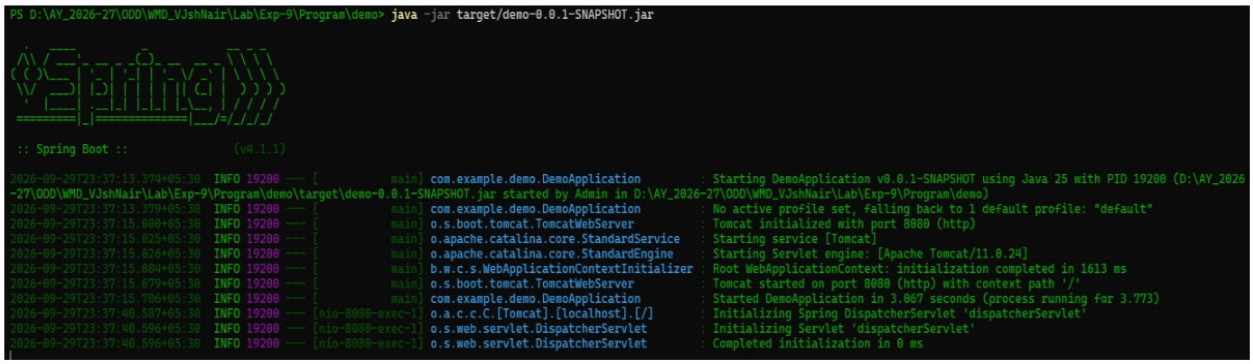
Step 6 — Run the Application

Running the generated .jar file with the java -jar command starts the application exactly as it would run on any server — this is the same command Render will effectively use once the application is

containerized in Part B.

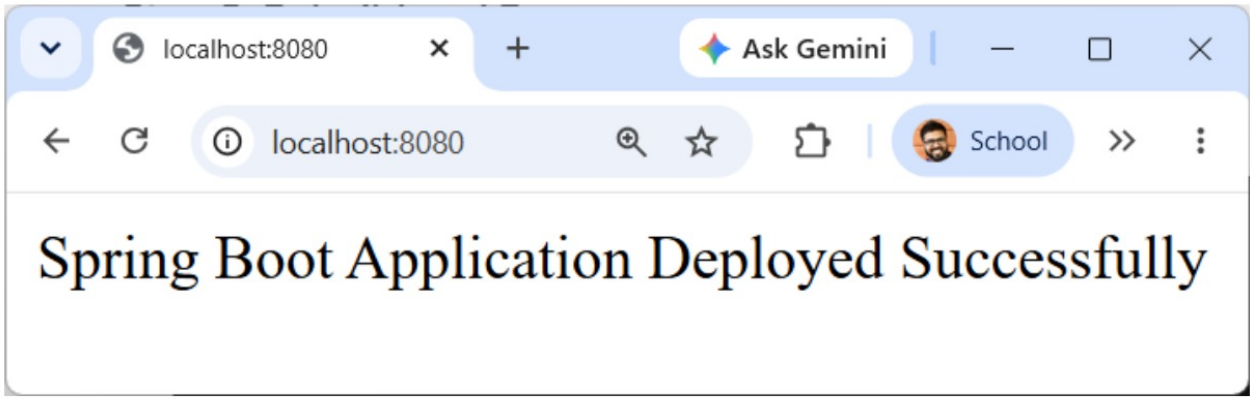
In the same terminal, execute:

```
java -jar target/demo-0.0.1-SNAPSHOT.jar
```



Step 7 — Access the Application

Open http://localhost:8080 in a browser to verify the deployment. The HelloController from Step 2 responds with:



This confirms that the local deployment is working correctly before moving on to the cloud deployment in Part B.

Part B — Deploy the Spring Boot Application on Render

Step 1 — Create a GitHub Repository and Push the Project

As explained in the Theory section, Render deploys applications by pulling code from a connected GitHub repository, rather than accepting a manual upload — so the very first step of cloud deployment is always getting the project onto GitHub.

Open PowerShell at D:\AY_2026-27\ODD\WMD_VJshNair\Lab\Exp-9\Program\demo\, initialize Git, and commit the project files:

```
git init
git add .
git commit -m "Initial commit for Render deployment"
```

Then, in a browser, go to GitHub, log in, click New Repository, name it demo, set its visibility to Public, and click Create repository.

Step 1A — Generate a Personal Access Token on GitHub

A Personal Access Token stands in for a password when pushing code from the command line.

1. Click your profile picture in the upper-right corner of any GitHub page, and select Settings.
2. Scroll to the bottom of the left-hand sidebar and click Developer settings.
3. In the new left sidebar, click Personal access tokens.
4. Choose Tokens (classic) for this experiment.
5. Click Generate new token → Generate new token (classic).
6. In Note, enter a name (e.g. RenderLab).
7. Under Select scopes, check the repo checkbox — this grants repository permissions.
8. Scroll to the bottom and click Generate token.
9. Copy the token immediately (it looks like ghp_xxxxxxxxxxxxxxxxxxxx) — GitHub will not show it again.

Step 1B — Update the Git Remote with the Token

In the PowerShell terminal, run (replacing <YOUR_TOKEN> with the token just copied):

```
git remote set-url origin https://<YOUR_TOKEN>@github.com/vijesh10-test7/demo.git
```

Step 1C — Push to GitHub

```
git push -u origin main
```

Step 2 — Create a Render Account

- Go to https://render.com.
- Click GET STARTED FOR FREE or Log In.
- Choose GitHub to sign in directly with the GitHub account used above.

Step 3 — Create a New Web Service on Render

- On the Render Dashboard, click New + → Web Service.
- Select Build and deploy from a Git repository → click Next.
- Connect the GitHub account if prompted, then find and click Connect next to the demo repository.

Step 4 — Configure Deployment Settings (Dockerfile)

As explained in the Theory section, Render's Docker deployment method needs a Dockerfile to know exactly how to build and run the application. What this code does: it defines a two-stage build — the first stage compiles the project using Maven and the full JDK, and the second stage copies only the resulting .jar file into a minimal Java runtime image.

What comes next: once this file is committed and pushed to GitHub, Render will automatically detect it and use it to build the container image — no manual build or start command needs to be typed into the Render dashboard.

> *Why this file is important: without a Dockerfile (or an equivalent build configuration), Render has no way of knowing this is a Java project that needs to be compiled with Maven before it can be run — the Dockerfile is the single source of truth for how the deployed container is built.*

In the project directory (D:\AY_2026-27\ODD\WMD_VJshNair\Lab\Exp-9\Program\demo\), create a new file named Dockerfile (no file extension):

D: > AY_2026-27 > ODD > WMD_VJshNair > Lab > Exp-9 > Program > demo > Dockerfile >

```
# Step 1: Build stage
FROM maven:3.9-eclipse-temurin-17 AS build
WORKDIR /app
COPY . .
RUN chmod +x ./mvnw
RUN ./mvnw clean package -DskipTests

# Step 2: Run stage
FROM eclipse-temurin:17-jre
WORKDIR /app
COPY --from=build /app/target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Step 4A — Push the Dockerfile to GitHub

In the PowerShell terminal at the project directory, run:

```
git add Dockerfile
git commit -m "Add Dockerfile for Render deployment"
git push origin main
```

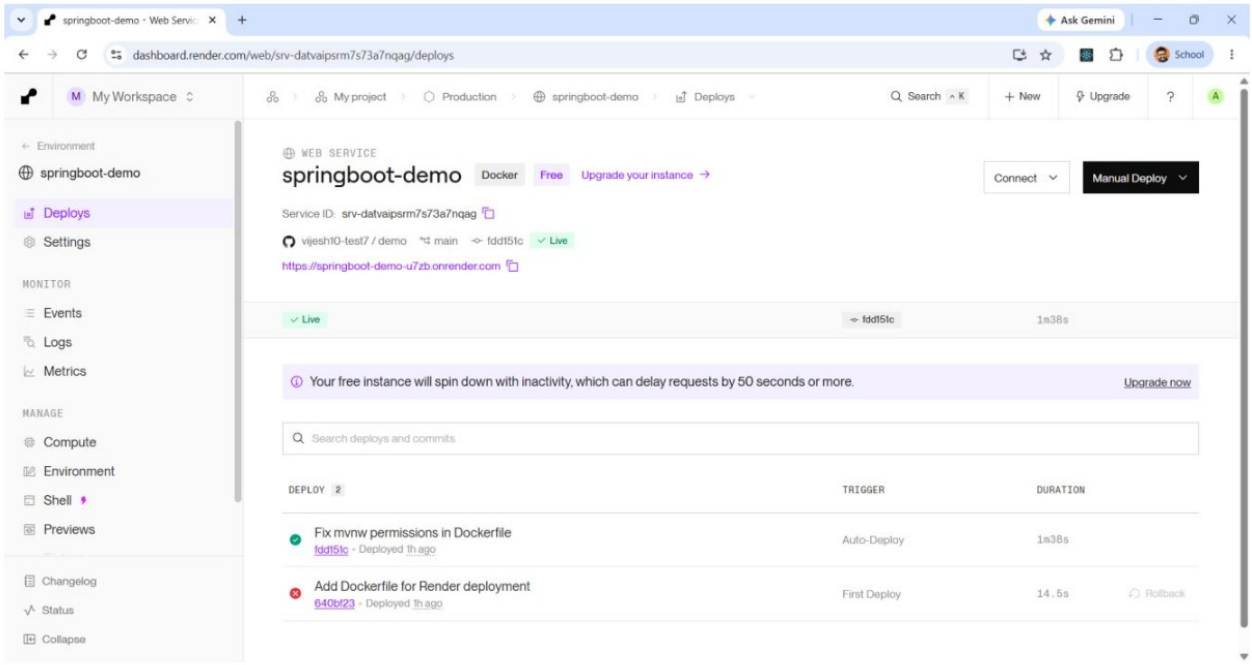
Step 4B — Complete the Render Setup

Back in the browser on Render, fill in the fields:

Field	Selection / Value
Name	springboot-demo
Language	Docker
Branch	main
Instance Type	Free

When Docker is selected, Render automatically uses the Dockerfile in the repository to build and run the container, so no manual build/start commands are needed.

Click Create Web Service at the bottom to launch the deployment.

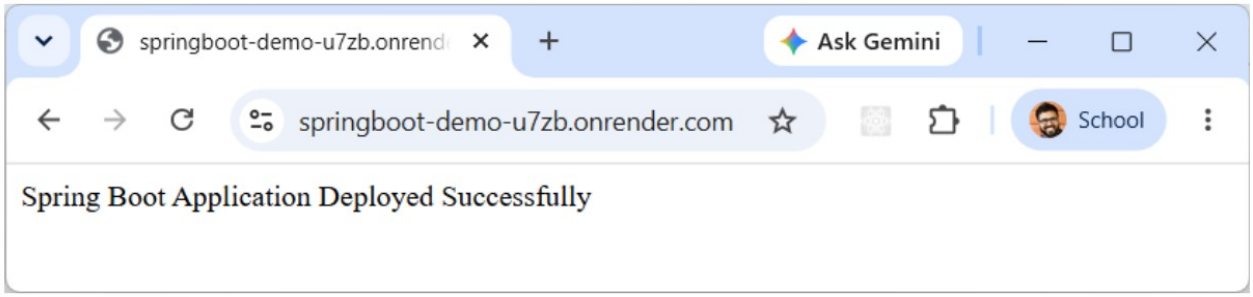


Step 5 — Access the Deployed Application

Once the build finishes, the application is reachable at the URL Render assigns to it, for example:

```
https://springboot-demo-u7zb.onrender.com
```

Visiting this URL should show the same confirmation message produced by HelloController.java in Part A:



This confirms that the application has been successfully containerized, deployed, and is publicly accessible over the internet.

Conclusion: -

Thus, we have demonstrated and Deploy Spring Boot app to a cloud platform (Render/Heroku) or local server.